

## MODULE-12

### VERSION CONTROL – GIT

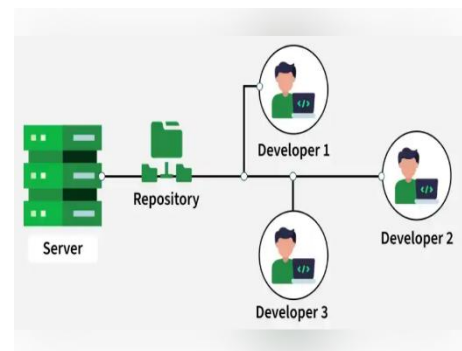
#### VERSION CONTROL SYSTEMS

A Version Control System (VCS) is a tool used in software development and collaborative projects to track and manage changes to source code.

- Tracks and records changes to the codebase.
- Enables multiple developers to collaborate without conflicts.
- Stores project files in a shared repository.
- Allows restoring previous versions when needed.

#### Components:

1. **Repository:** Stores project files and their change history.
2. **Revision:** A saved version of a file or project.
3. **Branch:** A separate copy of the code for independent development.
4. **Merging:** Combines changes from one branch into another.
5. **Commit:** A snapshot of changes saved to the project history.



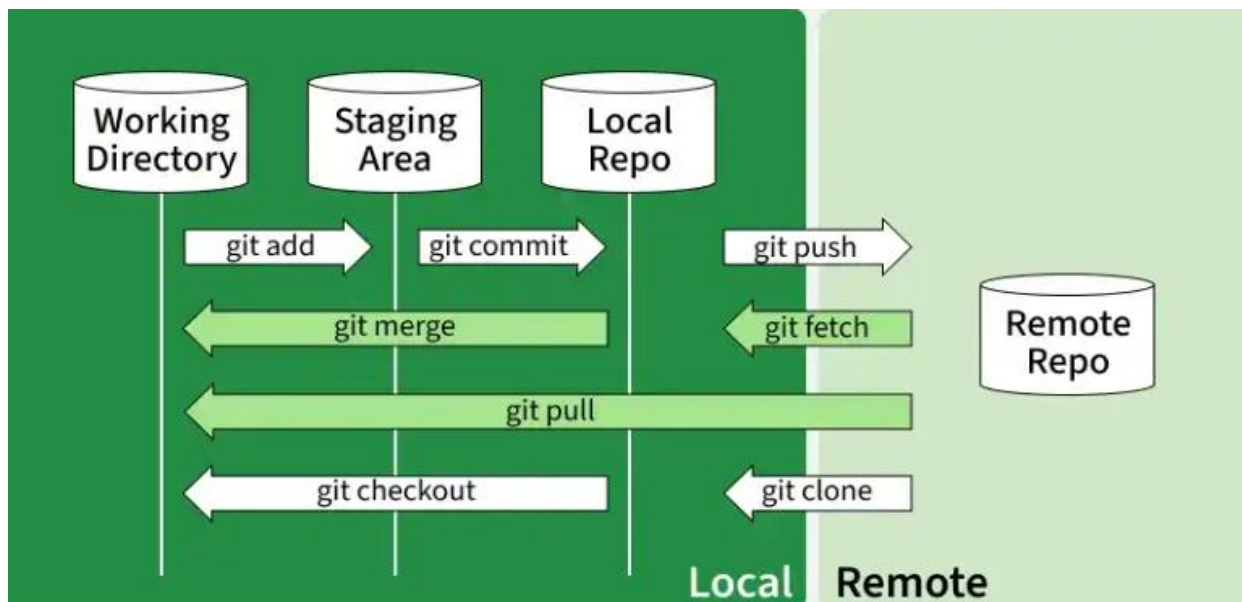
#### Types of Version Control Systems:

1. **Local Version Control Systems:** stores all project versions on a single computer and is mainly used by one user without remote collaboration.
  - No internet or server dependency.
  - Useful for individual projects.
  - Limited to single-user environments.
2. **Centralized VCS:** all the files and their version history are stored in a single central server. Developers connect to this server to access or modify files.
  - Developer retrieves the latest version of the files from the central server.
  - The developer works on the files.
  - Developer saves (commits) the changes directly back to the central server, making them immediately available to everyone else.

3. **Distributed VCS:** allows each developer to have a local repository along with a working copy of the project. Changes made in the local repository are not automatically visible to others.
- **Commit:** Saves changes to the local repository, visible only to the developer.
  - **Push:** Uploads committed changes to the central/shared repository so others can access them.
  - **Pull:** Downloads changes from the central repository to the local repository.

## GIT

Git is an open-source distributed version control system that helps teams track and manage code changes, collaborate seamlessly and work on projects of any size.



### Three Stages:

#### 1. Working Directory:

- The project folder where you create and modify files.
- Changes made here are not yet tracked by Git.

#### 2. Staging Area:

- A temporary area that holds selected changes before committing.
- Use `git add` to move changes here.

#### 3. Repository:

- Stores the committed versions (snapshots) of the project.
- Use `git commit` to save staged changes permanently.

## Installing Git

Download Git from the official website and install it by following the setup wizard.

### Configuring Git:

1. Setting username and email using:
  - **git config --global user.name "Your Name"**
  - **git config --global user.email "you@example.com"**
2. Initialising a New Repository:
  - Create a new Git repository in a project using **git init**.
3. Cloning an Existing Repository:
  - Copy an existing remote repository to your local system using:  
**git clone <repository-url>**

### Basic GIT Commands

1. **To check version of Git:** `git --version`
2. **Move into folder:** `cd <file path>` command
3. **Initialize Git Repository:** `git init`
4. **Current status of Repository:** `git status`
5. **Stage file for commit:** `git add .` or `git add <file-name>`
6. **Committing Changes:** `git commit -m "commit message"`
7. **Push changes to GitHub:** `git push origin <branch-name>`
8. **Make a Pull Request:** `git pull origin <branch-name>`
9. **Creates new branch:** `git branch <branch-name>`
10. **Switch to specified branch:** `git checkout <branch-name>`
11. **Merge change from given branch to current one:** `git merge <branch-name>`
12. **Displays commit history:** `git log`

### Branching

A branching strategy defines how developers create, manage and merge branches in a version control system like Git to ensure smooth collaboration and organized code development.

1. Create a Branch:
  - `git branch new-feature`
2. Navigate to the branch:

- `git checkout new-feature` or `git checkout -b new-feature`
3. Check current Branch:
    - `git branch`
  4. Delete a branch:
    - `git branch -d <branch-to-delete>`

## Merging

Git Merge combines changes from different branches into a single branch, integrating work while preserving history. It helps unify development without losing progress.

- Preserves commit history and creates a merge commit when needed.
  - Performs fast-forward or automatic merges unless conflicts occur.
  - Commonly used to integrate feature branches into the main branch.
1. Switch to the Target Branch:
    - `git checkout <target-branch>`
  2. Pull the Latest changes:
    - `git pull origin <target-branch>`
  3. Merge the feature-branch:
    - `git merge <feature-branch>`
  4. Test the Merged Code
  5. Push the change:
    - `git push origin <target-branch>`

## Types of Merging:

1. **Fast Forward merging:** Occurs when Git advances the current branch pointer to the latest commit without creating a merge commit.
  - Happens when the target branch is directly ahead of the current branch.
  - Git moves the branch pointer forward instead of creating a new commit.
  - Fast-forward merging only possible when branches have not diverged.
  - When the target branch has no new commits since branching.
2. **Three-way merging:** Occurs when branches have diverged, so Git creates a new merge commit by combining their changes.
  - Happens when the base branch has new commits after branching.
  - Git compares both branches with a common base commit.
  - A new merge commit is created to combine changes.

## Repositories

A Git repository (or repo) is a storage space where your project's files and their complete history of changes are kept. It allows you to track, manage, and collaborate on code over time.

- Stores all project files, commits, branches, and history.
- Maintains version history and allows restoring previous versions.
- Supports collaboration among multiple developers.
- Can be cloned to other systems for development.

### 1. Local Repository:

- Stored on your own computer.
- Allows you to make changes, commit them, and review your project history without needing an internet connection.
- Example: The .git folder inside your project contains the local repository.

### 2. Remote Repository:

- Hosted on a server like GitHub, GitLab, or Bitbucket.
- Enables multiple developers to collaborate on the same project.
- Supports operations like push, pull, and fetch to synchronize changes with the local repository.

## Based on Structure:

### 1. Bare Repository: Contains only the version history and Git data, no working files.

- Mainly used on servers for collaboration.
- Cannot directly edit files; only supports Git operations like push and fetch.
- Usually ends with .git extension.
- Acts as a central repository for multiple developers.

### 2. Non-Bare Repository: Contains working files along with the Git history.

- Used on local machines for development.
- Files can be edited directly; supports all Git operations including commit and merge.
- Does not usually end with .git extension.
- Acts as a local copy for development and testing.

## Git Push And Pull Commands

1. **Git Push:** This git push command is used to push all the commits of the current repository to the tracked remote repository.

- `git push -u origin master`
2. **Git Pull:** The `git pull` command updates your local repository by fetching and merging changes from the remote repository.
    - `git pull`

## **FORKING**

Git fork is a personal copy of another user's repository that allows you to experiment and make changes independently without affecting the original project.

- Creates an independent copy in your account.
- Includes all files, commit history, and branches.
- Commonly used for contributing to open-source projects.

### **Steps to Fork a Repository:**

1. Navigate to the Repository
2. Click on the fork button
3. Make changes and create a Pull Request

## **Workflows**

Git Flow is a branching model that involves the use of different types of branches based on the objective of the task.

### **GitFlow strategy:**

1. **Main:** This branch is used for production deployment.
2. **Develop:** New features are merged into this branch.
3. **Feature:** A branch was created to develop a new feature.
4. **Release:** This branch prepares code for a production release.
5. **Hotfix:** A branch for critical production fixes

### **Agile Development Lifecycle:**

1. **Plan / Requirement Gathering:** Define project goals, scope, and roadmap.
2. **Design:** Create UI wireframes and finalize the project design.
3. **Development:** Write, review, and test code based on user stories.

4. **Testing:** Verify the application meets requirements and fix defects.
5. **Deployment:** Deploy the tested application to the target environment.
6. **Delivery:** Prepare and launch the final product for users.
7. **Maintenance:** Update and improve the product based on user feedback.